ENGINEERING LOG · METHODS & FAILURE ANALYSIS

Every approach tried, what broke, and how each was resolved

A companion record to the main report: the full path from a hung board on two SDKs to working CC310 entropy passing all thirteen NIST tests.

TOOLCHAINS TRIED	ENTROPY PATHS TRIED	DISTINCT DEFECTS	FALSE CONCLUSIONS
3	4	7	2
FINAL OUTCOME			
13 / 13 pass			

OVERVIEW

Where the time went

The objective was simple to state: get true random numbers out of the nRF52840's CryptoCell CC310 and prove statistically that they are random. The obstacle was that the CC310 hung on every attempt for a long stretch of the project, in a way that convincingly imitated broken silicon.

The root cause turned out to be a single missing initialisation call. Getting there involved eliminating three toolchains, four entropy paths, and a set of build and data-handling defects that had nothing to do with the chip. This document records all of them, including two occasions where the evidence was read wrongly.

SUMMARY

Methods attempted

APPROACHES, IN ORDER

APPROACH	OUTCOME
macOS + nRF Connect SDK 2.9.1	`ABANDONED` flashing unreliable
Windows + NCS 3.4.0, Zephyr samples	`WORKED` hello_world, blinky
CC310 via Zephyr PSA Crypto	`HUNG`
CC310 via raw `nrf_cc3xx_platform_entropy_get()`	`HUNG`
Nordic's own unmodified `crypto/rng` sample	`HUNG`
NCS 2.9.1 cross-version check	`HUNG`
Arduino + SoftDevice RNG pool	`WORKED`
CC310 via Adafruit nRFCrypto, direct call	`HUNG` missing init
CC310 via `nRFCrypto.begin()`	`WORKED`

DEFECT 01

Build paths with spaces and across drives

Zephyr build fails on the project path

`RESOLVED`

SYMPTOM

`ValueError: path is on mount 'C:', start on mount 'E:'`, then `CMake Error: File not found: E:/IIIT`.

CAUSE

Two separate issues: `west` cannot compute a relative path between drives, and Zephyr's CMake scripts split the path `E:\IIIT Dharwad\TRNG` at the space.

RESOLUTION

Build from a space-free directory on the same drive as the SDK. The same class of defect reappeared later with `arduino-cli`, fixed by relocating the toolchain and sketches to `E:\TRNG_arduino`.

LESSON

Embedded toolchains routinely mishandle spaces in paths. Keep build trees on short, space-free paths.

DEFECT 02

The CC310 hang, and two wrong conclusions

This was the central problem. The chip initialised successfully, then hung permanently on the first request for random bytes. No serial output, no USB enumeration, unrecoverable by reset.

Hypotheses tested and eliminated

ZEPHYR / NCS INVESTIGATION

HYPOTHESIS	TEST	RESULT
Bug in application code	Ran Nordic's unmodified `crypto/rng` sample	Same hang
Wrong PSA driver selected	Set `OBERON=n`, `CC3XX=y` per Nordic's own DK config	No change
IRQ handler never installed	Enabled `CONFIG_DYNAMIC_INTERRUPTS`	No change
CTR_DRBG conditioning layer at fault	Called raw entropy API, bypassing DRBG	Same hang
Bootloader soft-jump leaves bad state	Full cold power-on reset	Same hang
SDK-version regression	Fresh NCS 2.9.1 install, separate toolchain	Same hang
Known silicon erratum	Checked nRF52840 Rev 2 errata sheet	No RNG entries

Localising the hang without serial output

With no console available, execution was traced using the on-board LED. `SYS_INIT` hooks were registered at specific initialisation priorities to bracket the CryptoCell driver's own init calls, with further markers inside `main()`. Each stage set a distinct colour, so the last colour shown identified the last line reached.

This established the sequence precisely: GPIO init → CryptoCell early init → CryptoCell post-kernel init → `psa_crypto_init()` → **hang on first generate call**.

First wrong conclusion

MISDIAGNOSIS

CLAIM — "Hardware fault in the CC310 silicon."

BASIS — Reproduced on two SDK versions with Nordic's own sample; no applicable errata.

WHY WRONG — The evidence genuinely excluded application bugs and SDK regressions, but never tested the assumption that the initialisation sequence itself was complete. Every path tried shared the same unexamined premise.

Second wrong conclusion

MISDIAGNOSIS

CLAIM	"Confirmed across two independent software stacks, therefore hardware."
BASIS	The Arduino/Adafruit path hung identically to Zephyr, seemingly independent corroboration.
WHY WRONG	Both stacks were misused in the same way. Agreement between two tests that share a hidden assumption is not independent confirmation.

ACTUAL ROOT CAUSE

The CryptoCell hardware must be powered up by `SaSi_LibInit()` before any `CRYS_*` call. The test code constructed an `nRFCrypto_Random` and called its `begin()` directly, which runs `CRYS_RndInit()` but never `SaSi_LibInit()`.

`CRYS_RndInit()` performs software-only setup, so it reported **success** — then `CRYS_RND_GenerateVector()` blocked forever polling hardware that was never enabled.

Resolution: route through the library's top-level `nRFCrypto.begin()`, which performs both steps in the correct order. The CC310 produced output on the first attempt afterwards.

How it was found: by reading the library's own `examples/random/random.ino` and comparing it against the code under test. That example uses `nRFCrypto.begin()`. The comparison took two minutes and should have been the first step, not one of the last.

DEFECT 03

Abstract class, silent at first glance

`CC310NoiseSource` would not compile

RESOLVED

SYMPTOM	Class rejected as abstract.
CAUSE	`NoiseSource` declares two pure-virtual members, `stir()` and `calibrating()`. Only `stir()` had been overridden.
RESOLUTION	Added a `calibrating()` override returning `false`: the CC310 is a certified DRBG that is either initialised or not, with no ring-oscillator warm-up period to wait out.

DEFECT 04

Four-bit shift corrupting the captured stream

Capture script stitched truncated lines

RESOLVED

SYMPTOM	Serial test returned `0.010283`, fractionally above the 0.01 pass threshold. Everything else passed comfortably.
CAUSE	On a truncated line the script kept the leftover hex character and prepended it to the next line, displacing every subsequent byte by four bits.
WHY DANGEROUS	Shifted random data still looks random. Nothing crashes, no error appears, and the file silently stops representing what the hardware produced.
RESOLUTION	Truncated lines are discarded whole and counted. Losing one 128-byte chunk is preferable to corrupting every byte that follows. The same test then returned `0.028721`.

DEFECT 05

USB buffer overflow from oversized writes

Lines truncated during sustained streaming

MITIGATED

SYMPTOM	A 1 MB capture arrived 2,720 bytes short, with roughly one truncated line per 90,000 bytes.
CAUSE	256-byte chunks produce 513-character lines, which exceed the USB CDC transmit buffer in a single write.
RESOLUTION	Reduced to 128-byte chunks. Occasional whole-line loss still occurs under sustained streaming; because lines are now decoded independently this costs samples without biasing them, and the capture script reports the count.
STATUS	Accepted rather than eliminated: for statistical testing, missing samples are harmless where corrupted samples would not be.

DEFECT 06

Serial formatting as the throughput bottleneck

Capture far slower than the hardware

RESOLVED

SYMPTOM — Transfers took far longer than the CC310's generation rate could explain.

CAUSE — `Serial.print(byte, HEX)` invokes the formatting machinery once per byte — over a million calls per capture.

RESOLUTION — Hex-encode via a 16-entry lookup table into a buffer, then issue one bulk `Serial.write()` per line. Roughly 1 MB now transfers in 18 seconds.

DEFECT 07

A test suite that had to be trusted before use

The SP 800-22 implementation was written from scratch, which raises an obvious question: if the suite reports PASS, is that the data or a bug in the tests?

It was therefore run against four datasets with known expected outcomes before being applied to hardware output.

SUITE VALIDATION

DATASET	EXPECTED	OBSERVED
`os.urandom`	Pass all	3 / 3 fresh samples clean
All-zero bytes	Fail nearly all	11 / 13 failed
70%-biased bitstream	Fail nearly all	10 / 13 failed
Weak LCG	Fail spectral	8 / 13 failed, incl. DFT

An apparent failure that was not one EXPLAINED

SYMPTOM	The first `os.urandom` run failed two tests, though it should pass everything.
INVESTIGATION	Three freshly generated samples were tested. All three passed cleanly.
EXPLANATION	The suite performs roughly forty comparisons at α = 0.01, so a perfect source has close to a one-in-three chance of one spurious failure per run. This is the definition of α, not a defect.
CONSEQUENCE	Single isolated failures are never treated as meaningful. Only repeated failures, or the same test failing across independent captures, are.

SCOPE DECISION

Why the entropy pool version is not the evidence

The CC310 was also wired into the Southern Storm `RNGClass` pool as a `NoiseSource`, which works and passes all thirteen tests. Those figures are nevertheless not cited as hardware results.

`RNGClass` does not pass hardware bytes through: it folds them into a ChaCha20 state and returns keystream. A software stream cipher passes SP 800-22 even when fed a poor entropy source, so testing its output measures ChaCha20 rather than the CryptoCell. All hardware figures come from raw `CRYS_RND` output.

TAKEAWAYS

What this cost, and what it taught

- **Check the vendor's own working example first.** The defect that consumed most of the project was visible in a two-minute comparison against `examples/random/random.ino`.
- **A success return value is not proof of success.** `CRYS_RndInit()` reported success while the hardware sat unpowered. An API that validates only its own arguments cannot vouch for the peripheral.
- **Repeated agreement is not independent confirmation.** Two stacks hung identically because both were misused identically.
- **Prefer losing data to corrupting it.** The nibble-carry bug produced output that looked correct and was not; dropping whole lines is visibly lossy and therefore safer.
- **Validate the instrument before trusting the measurement.** Running the suite against known-bad data is what makes its PASS verdict meaningful.